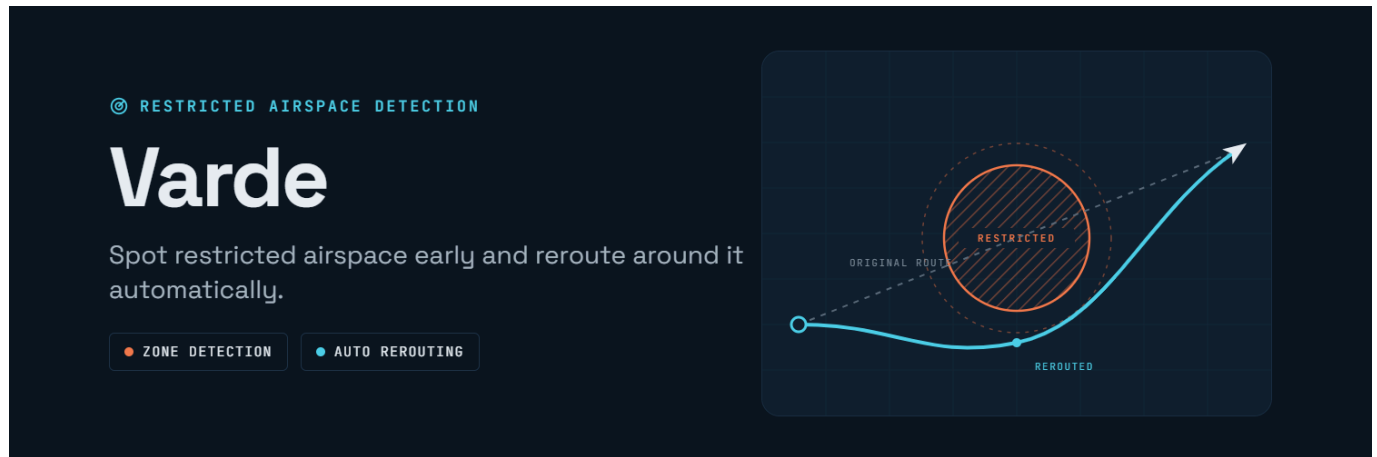


Software Requirements Specification (SRS)

Varde — Restricted Airspace Warning & Avoidance



- **Version:** 1.2
- **Status:** Draft
- **Standard target:** IEEE 830-style structure (adapted for course use)
- **Tech focus:** C++17 / Qt, Protobuf, Redis Streams, .NET 8 + PostGIS/TimescaleDB + OGC API – Features, Spring Boot 3 + Keycloak, Docker Compose, Linux
- **Companion policy:** None (see VD-DAT-002)
- **Delivery:** Thursday 29 October 2026 (morning)
- **Starting point:** This is a **brownfield** case. The team inherits an existing C++ flight-data processor (see 3.2.1).

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) defines the requirements for **Varde**, a restricted-airspace safety net that warns air traffic controllers *before* an aircraft enters a restricted area and, through its **Vektor** function, proposes a detour around it.

Nordavia Airspace Coordination (a fictional joint civil/military airspace cell on the west coast of Norway) manages military danger areas, restricted zones around sensitive sites, and temporary restrictions for events such as rescue operations. Today, air traffic controllers usually learn that an aircraft has flown into one *after* it happened, often by phone. Nordavia wants three things: a warning early enough for an air traffic controller to act, a proposed route around the area that an operator can approve with one click, and a complete record of who acknowledged and approved what.

The name comes from the *varde* — the stone cairns that marked dangerous places along the Norwegian coast. **Varde** raises the warning; **Vektor** proposes the way around.

1.2 Document Conventions

- **"Shall"** indicates a mandatory requirement. **"Should"** indicates a recommended requirement. **"May"** indicates an optional requirement.
- **Requirement IDs** follow the pattern **VD-<AREA>-###** (e.g. **VD-FUN-001**).
- Unless otherwise stated, requirements apply to the complete deployed system.
- **Consultants** refers to the students forming the project team.
- Coordinates are WGS84. In GeoJSON and PostGIS, coordinate order is **longitude, latitude**.

1.3 Intended Audience and Reading Suggestions

This document is intended for the consultant team and course staff. Consultants should:

1. Read Sections 1–2 to understand scope and constraints.
2. Read 3.2.1 (the inherited processor) before writing any code.
3. Implement requirements in Section 3, starting with the **core pipeline**: *a scripted flight heading toward an active restricted area raises a PREDICTED warning that appears on the Qt map within 1 second, and escalates to IMMINENT and INFRINGEMENT as the aircraft approaches and enters. The area itself is served by the Geo API.*
4. Use Sections 4–6 as quality gates for completion.

1.4 Product Scope

Varde is a **simulation-grade** system intended for educational demonstration of:

- geospatial storage and standards-based geospatial APIs (OGC API – Features),
- real-time geometric computation (path projection, polygon intersection, route planning),
- event-driven integration across three languages (C++, C#, Java) through one Protobuf contract,
- role-based authorisation with an external identity provider,
- a desktop operator console consuming live and historical data.

It is **not** intended for use in real air traffic management.

1.5 Glossary

Track A snapshot of one aircraft's state: ICAO24 identifier, timestamp, position, altitude, ground speed and ground track (heading over the ground).

Live track / simulated track A live track comes from OpenSky. A simulated track comes from a scenario file and has a known **destination**. Only simulated tracks can follow a detour.

Zone (restricted area) A polygon on the map that aircraft must not enter while it is **active**. Types: **DANGER** (e.g. military firing range), **RESTRICTED** (e.g. sensitive site), **TEMPORARY** (time-limited, e.g. a rescue operation).

Active zone A zone whose restriction is **in force right now**: aircraft may not fly into it, so Varde watches traffic around it, warns when an aircraft is heading in, and proposes detours. An **inactive** zone is still stored on the map, but aircraft may fly through it freely and Varde ignores it.

Example. A military firing range off the coast is closed only while an exercise is running. During the exercise the zone is active: an aircraft heading for it gets warnings and a detour proposal. Outside the exercise it is inactive: the same area, but planes fly over it normally and Varde raises nothing.

Whether a zone is active is decided by two settings:

- **Enabled:** an on/off switch a supervisor controls from the console (VD-FUN-045).
- **Activation window (optional):** a from-to time, for zones that only apply for a set period (e.g. the exercise, or a rescue operation from 10:00 to 16:00).

A zone is active when it is **enabled** and, if it has a window, **the current time is inside it**. A zone with no window is active whenever it is enabled. So a supervisor can switch off a rescue zone early when the operation ends, even though its window is still open.

Avoidance area A zone enlarged by its safety margin. Detours are planned around avoidance areas, not around the bare zones.

Look-ahead How far ahead each track's path is projected: **120 s**.

Projected path The straight segment from a track's current position to where it will be after the look-ahead, at its current ground speed and ground track.

Time to entry The time until a track's projected path first crosses into an active zone.

Warning levels (computed per track-zone pair every cycle)

- **NONE** — no entry projected within the look-ahead.
- **PREDICTED** — entry projected within 120 s.
- **IMMINENT** — entry projected within 30 s.
- **INFRINGEMENT** — the track is inside the zone now.

Alert The record of one track-zone pair while its warning level is above NONE. An alert is **open** from its first level above NONE until it **closes** (see VD-FUN-024).

Vektor proposal A detour — a list of waypoints — that takes a simulated track around an avoidance area to its destination.

Proposal states

- **PROPOSED** — waiting for an operator.
- **APPROVED** — an operator accepted it; the simulated track follows it.
- **REJECTED** — an operator declined it.
- **EXPIRED** — not decided in time, superseded, or no longer needed.

Cycle One pass of the processor's main loop (500 ms): ingest tracks, compute warnings, compute proposals, publish.

Inherited processor The C++ processor from a previous CapOps team, forked by this team (3.2.1).

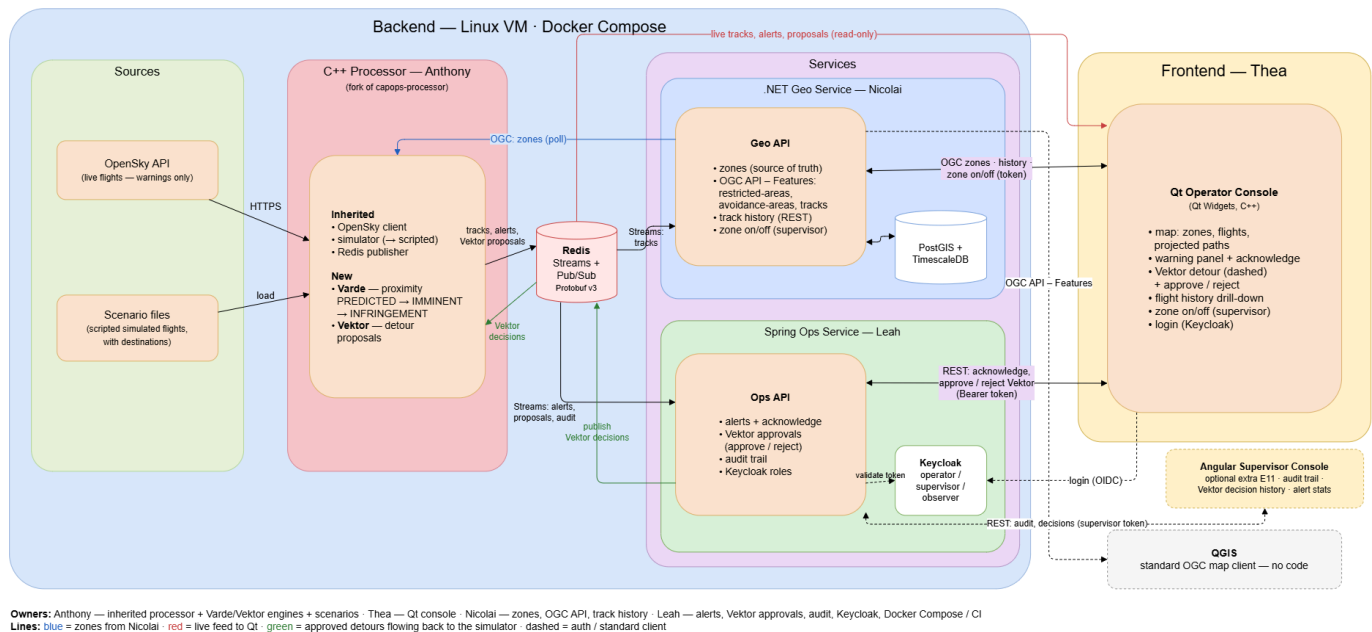
2. Overall Description

2.1 Product Perspective

Varde is composed of:

- **Sources**
 - OpenSky Network API (live tracks; warnings only)
 - Scenario files (scripted simulated tracks with destinations)
- **Processor** (C++, inherited and extended) — ingests tracks, fetches zones, runs the **Varde** proximity engine and the **Vektor** detour engine, publishes events
- **Redis** — event transport (Streams, plus the inherited Pub/Sub channel)
- **Geo Service** (.NET) — source of truth for zones; stores track history in PostGIS/TimescaleDB; serves OGC API – Features and a history API
- **Ops Service** (Spring Boot) — alerts, acknowledgement, Vektor approval workflow, audit trail
- **Keycloak** — login and roles
- **Qt Operator Console** — the only user interface built by the team
- **QGIS** — standard desktop GIS used to verify the OGC API (no code)

Suggested architecture:



The authoritative architecture diagram is maintained by the team (VD-DOC-003).

2.2 Product Functions (High Level)

Varde shall:

- ingest live and simulated tracks,
- serve restricted areas as standard geospatial data,
- project every track's path and raise escalating warnings before it enters an active zone,
- propose detours for simulated tracks and let an operator approve or reject them,
- make approved detours take effect in the simulation,
- record every acknowledgement, approval, rejection and zone change with who and when,
- store track history and serve it for drill-down and replay,
- present all of this in a Qt operator console, with different capabilities per role.

2.3 User Classes and Characteristics

Role	Needs	Interface
Operator	See tracks, zones and warnings; acknowledge warnings; approve or reject Vektor proposals	Qt console
Supervisor	Everything an operator can do, plus enable/disable zones and read the audit trail	Qt console
Observer	Read-only view of the same picture (training, visiting staff)	Qt console
System Services	Processor, services; no human interaction	—

Roles are **real Keycloak accounts**. Permissions shall be enforced by the services, not only hidden in the UI.

2.4 Operating Environment

- Processor and Qt console: C++17, CMake, Qt 6, Linux or macOS (the inherited CI builds on both); Windows optional
- Geo Service: .NET 8, Npgsql + NetTopologySuite
- Ops Service: Java 21, Spring Boot 3
- Database: PostgreSQL with PostGIS and TimescaleDB (e.g. the [timescale/timescaledb-ha](#) image)
- Redis 7, Keycloak (Docker)
- Deployment: Docker Compose, runnable locally (deploying to a Linux VM over SSH is expansion E12)
- QGIS (any recent version) for OGC verification

2.5 Design and Implementation Constraints

- The processor shall be the team's fork of the inherited processor (3.2.1), in **C++17** with **CMake**.
- The operator console shall be built from scratch in **Qt Widgets (C++)**. No code from previous CapOps frontends shall be used verbatim.
- The Geo Service shall be **C# / .NET 8**. The Ops Service shall be **Java / Spring Boot 3**.
- Services shall exchange events through **Redis** using **Protobuf** messages defined in **one shared .proto source**, generated for C++, C# and Java at build time.
- Consumers that must not miss events (Geo Service, Ops Service) shall use **Redis Streams** with consumer groups.
- Zones and track history shall be stored in **PostgreSQL with PostGIS and TimescaleDB**.
- The Geo Service shall implement **OGC API – Features Part 1 (Core)** with GeoJSON output.
- Authentication shall use **Keycloak** (OpenID Connect).
- The backend shall run with **Docker Compose** (VD-CI-003). Deploying it to a Linux VM is expansion E12.
- **Code review:** every merge request shall be reviewed by at least one teammate. Each consultant shall review at least one merge request per week in a component they do not own.

2.6 Assumptions and Dependencies

- OpenSky is rate-limited and not always available. **Scenario mode is the default** and every demo scenario shall run without internet access. Live mode is a configuration switch (inherited).

- Live tracks have no known destination, so they receive warnings but never Vektor proposals.
- Distances are computed with a local flat-earth approximation (VD-FUN-021). Geodesic accuracy is out of scope.
- Zones are 2D: altitude is ignored. An aircraft flying far above a low-altitude zone will still be warned. This known limitation shall be recorded in the decision log (expansion E4 removes it).
- Zone names and data are fictional.

3. Specific Requirements

3.1 External Interface Requirements

3.1.1 Event Transport (Redis + Protobuf)

VD-IF-001 (Shared Contract) All events shall be defined in one `.proto` source in a shared location, generated for C++, C# and Java during each component's build.

VD-IF-002 (Schema Version) The inherited code already attaches a small `MetadataProto` message to every event: the **contract version** it was built with and a **timestamp**. Every event shall keep carrying it. The previous team's contract is version 2. Because this project changes the contract (removing weather fields, adding alerts and Vektor proposals), events shall now carry **version 3**.

This is the version of *Varde's event contract*, not of the Protobuf language: the `.proto` file stays `syntax = "proto3"` throughout.

VD-IF-003 (Schema Evolution Rules) Fields shall never be renumbered or re-typed. Removed fields shall be marked `reserved`. New fields shall use new numbers. Consumers shall ignore unknown fields. A consumer generated from the **version 2** schema shall still parse a version 3 track snapshot (tested in VD-TST-008).

VD-IF-004 (Event Types) At minimum:

Event	Producer	Content
Track snapshot (inherited <code>FlightDataProto</code>)	Processor	All current tracks, every cycle
<code>ZoneAlertEvent</code>	Processor	Alert ID, ICAO24, zone ID, level, time to entry, event type (OPENED / LEVEL_CHANGED / CLOSED)
<code>VektorProposalEvent</code>	Processor	Proposal ID, alert ID, ICAO24, zone ID, waypoints, extra distance (km), extra time (s)
<code>VektorDecisionEvent</code>	Ops Service	Proposal ID, decision (APPROVED / REJECTED / EXPIRED), decided by, time
<code>AlertAcknowledgedEvent</code>	Ops Service	Alert ID, acknowledged by, time, level acknowledged
<code>AuditEvent</code>	Geo Service, Ops Service	Actor, role, action, target, time

VD-IF-005 (Streams) Events shall be written to these streams (names may be changed if documented):

[varde:tracks](#), [varde:alerts](#), [varde:vektor-proposals](#), [varde:vektor-decisions](#), [varde:acks](#), [varde:audit](#).

Streams shall be capped (e.g. [MAXLEN ~ 10000](#)).

VD-IF-006 (Inherited Channel) The processor shall keep publishing track snapshots on the inherited Pub/Sub channel [flightdata](#) for backward compatibility.

3.1.2 Geo Service API

VD-IF-010 (OGC Landing and Conformance) [GET /](#), [GET /conformance](#), [GET /collections](#), [GET /collections/{id}](#) per OGC API – Features Part 1.

VD-IF-011 (Collections) At minimum:

Collection	Content
restricted-areas	All zones, with type, enabled flag, activation window and active (computed)
avoidance-areas	Active zones enlarged by their safety margin
tracks	Stored track positions (point features with time)
coastline	A rough outline of the land for the map background: simplified real data (e.g. Natural Earth, public domain, clipped to the region) or a made-up outline. Accuracy doesn't matter

VD-IF-012 (Items) In OGC API terms, a **collection** is one dataset (like a table) and its **items** or **features** are the things in it (like rows): one zone, or one aircraft position. Each feature has a **geometry** (where it is: a polygon or a point) and **properties** (everything else). So [GET /collections/restricted-areas/items](#) returns the zones, [GET /collections/tracks/items](#) returns stored aircraft positions, and so on, all in the same format.

[GET /collections/{id}/items](#) shall return a GeoJSON [FeatureCollection](#) and support these query parameters:

Parameter	Meaning
bbox=minLon,minLat,maxLon,maxLat	Only features inside this rectangle of the map (map viewers send the visible area)
limit	Features per page: default 100, maximum 1000
datetime	tracks only: one instant, or an interval from/to

When there are more features than fit on one page, the response shall include a [next](#) link to the following page, plus [numberMatched](#) (total found) and [numberReturned](#) (on this page). [GET /collections/{id}/items/{featureId}](#) shall return one feature.

Example. GET /collections/restricted-areas/items?bbox=4.5,59.5,5.5,60.5&limit=10 returns the zones inside that rectangle (here one, the seed zone from Appendix A):

```
{
  "type": "FeatureCollection",
  "numberMatched": 1, "numberReturned": 1,
  "features": [{
    "type": "Feature", "id": "D-101",
    "geometry": { "type": "Polygon",
      "coordinates": [[[5.00,59.95],[5.20,59.95],[5.20,60.05],[5.00,60.05],
[5.00,59.95]]] },
    "properties": { "name": "Hjelteskjær Firing Range", "type": "DANGER",
      "enabled": true, "activeFrom": null, "activeTo": null,
      "active": true, "marginNm": 3.0 }
  }],
  "links": [{ "rel": "self", "href": ".../items?bbox=4.5,59.5,5.5,60.5&limit=10" }]
}
```

Coordinates are **longitude first** (VD-DAT-003), and the polygon's last corner repeats the first to close it.

VD-IF-013 (History API) GET /api/tracks/{icao24}/history?from=&to= shall return one aircraft's positions over a time interval.

VD-IF-014 (Zone Control) PATCH /api/zones/{id} with { "enabled": true|false } shall require the **Supervisor** role.

VD-IF-015 (Errors) Invalid parameters shall return 400 with an OGC-style JSON exception body. Missing or invalid tokens on protected endpoints return 401; insufficient role returns 403.

3.1.3 Ops Service API

VD-IF-020 (Endpoints) At minimum:

Method	Path	Role
GET	/api/alerts?state=open	any authenticated
POST	/api/alerts/{id}/acknowledge	Operator, Supervisor
GET	/api/vektor/proposals?state=PROPOSED	any authenticated
POST	/api/vektor/proposals/{id}/approve	Operator, Supervisor
POST	/api/vektor/proposals/{id}/reject	Operator, Supervisor
GET	/api/audit?from=&to=	Supervisor

VD-IF-021 (Late Decisions) Approving or rejecting a proposal that is no longer PROPOSED shall return 409 Conflict.

3.1.4 User Interface

VD-UI-001 (Qt Operator Console) A Qt Widgets desktop application, described in 3.2.6.

VD-UI-002 (QGIS) The [restricted-areas](#), [avoidance-areas](#), [tracks](#) and [coastline](#) collections shall load in QGIS as an OGC API – Features connection without any custom code.

3.2 Functional Requirements

3.2.1 Inherited Processor (Brownfield)

The team inherits [capops-processor](https://github.com/magnuax/capops-processor) (<https://github.com/magnuax/capops-processor>), built in March 2026 by a previous CapOps team. It is C++17 with CMake, Qt Core/Network, [redis-plus-plus](#), Protobuf and Catch2, with an OpenSky client, simulated flight and weather sources, a sector-capacity computation, a Redis publisher, and GitHub Actions CI/CD. **Only the processor is inherited.** No other code from previous teams shall be used verbatim.

VD-LEG-001 (Fork and Credit) The team shall fork the repository, keep its **GPL-3.0** licence, and credit the original authors in the README.

VD-LEG-002 (As-Found Review) Before changing code, the team shall write a one-page **as-found architecture note** (modules, data flow, build, what is tested), reviewed by a teammate who does not own the processor.

VD-LEG-003 (Characterisation Tests First) Before modifying any inherited module, the team shall add **characterisation tests** that pin its current behaviour, so refactoring cannot change behaviour unnoticed.

VD-LEG-004 (Remove What Is Not Needed) Weather sources and sector-capacity logic are not part of Varde. The team shall remove or disable them safely, mark their Protobuf fields [reserved](#) (VD-IF-003), and record the decision.

VD-LEG-005 (Scripted Simulation) The inherited random simulation (random flights that wrap around the map edges) shall be replaced by the scenario simulation in 3.2.3, **behind the inherited ITrackSource interface.**

VD-LEG-006 (Re-point Delivery) Inherited CI/CD shall be reconfigured to the team's own registry (and VM, if E12 is chosen). No credentials or targets from the previous team shall remain.

3.2.2 Zones (Geo Service)

VD-FUN-001 (Zone Model) Each zone shall have: ID, name, type (DANGER / RESTRICTED / TEMPORARY), polygon (WGS84), enabled flag, optional activation window (from/to, UTC), and safety margin in nautical miles (default **3 NM** = 5.556 km).

VD-FUN-002 (Seed Data) Zones shall be loaded from a GeoJSON seed file on first start. The seed file shall contain at least **6 zones** in the operating region (59–61°N, 3–7°E), including at least one of each type and at least one TEMPORARY zone with an activation window.

VD-FUN-003 (Active Computation) A zone shall be active when both of these are true:

- it is **enabled**, and

- it has **no activation window**, or the current time is **inside** its window.

Each feature in the [restricted-areas](#) collection shall include this result as an [active](#) property ([true](#) or [false](#)).

VD-FUN-004 (Avoidance Areas) [avoidance-areas](#) shall contain each active zone enlarged by its margin, computed in PostGIS (e.g. [ST_Buffer](#) on [geography](#)).

VD-FUN-005 (Zone Changes Audited) Every enable/disable shall publish an [AuditEvent](#) (actor, role, zone, old → new, time).

3.2.3 Processor — Sources and Zones

VD-FUN-010 (Scenario Files) Scenarios shall be files (JSON or YAML) listing simulated tracks, each with ICAO24, start position, destination, ground speed, and start delay. The same scenario file shall always produce the same tracks (deterministic).

VD-FUN-011 (Required Scenarios) At minimum:

Scenario	Purpose
S1 — Straight entry	One track flies straight into an active zone: PREDICTED → IMMINENT → INFRINGEMENT
S2 — Near miss	One track passes close to a zone without its path crossing it: no warning
S3 — Detour	One track whose direct route crosses a zone, with a destination beyond it: Vektor proposal
S4 — Load	150 tracks across the region
S5 — Zone switched on	A TEMPORARY zone activates while a track is already inside it: INFRINGEMENT at once

VD-FUN-012 (Zone Fetch) The processor shall fetch active zones and avoidance areas from the Geo Service's OGC API at start-up and every **30 s** (configurable).

VD-FUN-013 (Zone Fallback) If the Geo Service is unreachable, the processor shall keep using the last zone set it fetched (or the seed file on first start), and log that it is doing so.

3.2.4 Processor — Varde Proximity Engine

VD-FUN-020 (Projection) Every cycle, for every track, the processor shall compute the projected path: the segment from the current position to the position after **120 s** at current ground speed along current ground track.

VD-FUN-021 (Distance Model) Positions shall be converted to local kilometres with:

- 1° of latitude = **111.32 km**
- 1° of longitude = **111.32 × cos(latitude) km**, using the track's current latitude
- 1 knot = **1.852 km/h**

VD-FUN-022 (Warning Level) For every track and every active zone, the computed level for this cycle shall be:

- **INFRINGEMENT** if the current position is inside the zone,
- else **IMMINENT** if the projected path enters the zone with time to entry $\leq 30\text{ s}$,
- else **PREDICTED** if it enters with time to entry $\leq 120\text{ s}$,
- else **NONE**.

Time to entry = distance along the projected path to the first boundary crossing $\div$ ground speed.

Worked example — level from time to entry. Zone **D-101** (DANGER): 59.95–60.05°N, 5.00–5.20°E. A track at **60.00°N, 4.60°E**, ground track **090°** (due east), **360 kt**. At 60.00°N, 1° longitude = $111.32 \times \cos 60^\circ = 111.32 \times 0.5 = 55.66\text{ km}$. Speed = $360 \times 1.852 = 666.72\text{ km/h} = 0.1852\text{ km/s}$.

Track longitude	Distance to zone edge (5.00°E)	Time to entry	Level
4.6000°	$0.4000 \times 55.66 = 22.264\text{ km}$	$22.264 / 0.1852 = 120.2\text{ s}$	NONE (just outside 120 s)
4.6017° (one cycle later)	22.171 km	119.7 s	PREDICTED
4.8900°	$0.11 \times 55.66 = 6.123\text{ km}$	33.1 s	PREDICTED
4.9100°	$0.09 \times 55.66 = 5.009\text{ km}$	27.0 s	IMMINENT
5.0100°	inside	—	INFRINGEMENT

The same track at **60.10°N** flying due east never enters D-101 (its northern edge is 60.05°N): **NONE** throughout. That is scenario S2.

VD-FUN-023 (Hysteresis) The published level shall **rise immediately** when the computed level rises. It shall **fall only after 3 consecutive cycles** in which every computed level is lower than the published level, and then fall to the highest level computed in those 3 cycles.

Worked example — hysteresis.

Cycle	Computed	Published	Note
1	PREDICTED	PREDICTED	alert OPENED
2	IMMINENT	IMMINENT	rises immediately
3	PREDICTED	IMMINENT	lower, 1 of 3
4	NONE	IMMINENT	lower, 2 of 3
5	PREDICTED	PREDICTED	lower, 3 of 3 → falls to highest of cycles 3–5
6	NONE	PREDICTED	1 of 3
7	NONE	PREDICTED	2 of 3
8	NONE	NONE	3 of 3 → alert CLOSED

VD-FUN-024 (Alert Lifecycle) An alert shall be **OPENED** when a track–zone pair's published level first rises above NONE, and **CLOSED** when it falls back to NONE (per VD-FUN-023) or the track disappears. The processor shall publish a [ZoneAlertEvent](#) on OPENED, on every published level change, and on CLOSED.

VD-FUN-025 (Zone Deactivation) When a zone stops being active, every open alert for it shall be CLOSED at the next zone refresh.

3.2.5 Processor — Vektor Detour Engine

VD-FUN-030 (When to Propose) The processor shall create a Vektor proposal when a **simulated** track's alert reaches PREDICTED or IMMINENT and the direct segment from its position to its destination crosses the zone's avoidance area. Live tracks shall never receive proposals.

VD-FUN-031 (Detour Geometry) The detour shall go around the **convex hull** of the avoidance area, on the side with the shorter total path, using the hull vertices between the tangent points seen from the track and from the destination. The detour shall not pass through the avoidance area's interior (it may run along its boundary).

Worked example — choosing the side. For illustration the avoidance area is the rectangle **59.90–60.10°N, 4.90–5.30°E** (D-101 enlarged by ≈ 3 NM, simplified). Track **P** at 60.02°N, 4.60°E; destination **D** at 60.02°N, 5.60°E. The direct path along 60.02°N crosses the area. Local km, using $\cos 60^\circ$ for longitude (55.66 km/°) and 111.32 km/° latitude, with P at (0, 0): D = (55.66, 0).

North: P → N1 (4.90°E, 60.10°N) = (16.698, 8.906) → N2 (5.30°E, 60.10°N) = (38.962, 8.906) → D |P N1| = $\sqrt{16.698^2 + 8.906^2}$ = **18.925 km**; |N1 N2| = **22.264 km**; |N2 D| = **18.925 km** → total **60.11 km**

South: P → S1 (4.90°E, 59.90°N) = (16.698, -13.358) → S2 (5.30°E, 59.90°N) → D |P S1| = $\sqrt{16.698^2 + 13.358^2}$ = **21.384 km**; |S1 S2| = **22.264 km**; |S2 D| = **21.384 km** → total **65.03 km**

North is shorter → proposal waypoints **N1, N2**, extra distance 60.11 – 55.66 = **4.45 km**, extra time at 360 kt = 4.45 / 0.1852 $\approx$ **24 s**.

VD-FUN-032 (One Proposal per Alert) At most one PROPOSED proposal shall exist per alert. After a rejection, no new proposal shall be made for that alert. A new proposal for the same track supersedes the old one (which becomes EXPIRED).

VD-FUN-033 (Following an Approved Detour) On an APPROVED [VektorDecisionEvent](#), the simulated track shall fly to the proposal's waypoints in order and then to its destination. If the track no longer exists, the decision shall be logged and ignored.

3.2.6 Qt Operator Console

What the map actually is. The map is a plain canvas, not a real map. You don't need map tiles, a GIS library or accurate geography underneath. Everything on it comes from two sources:

- **Where the aircraft are and where they're heading** comes from the processor's live feed (real OpenSky or simulated tracks): positions as longitude/latitude, a heading and a projected path.
- **Where the zones are** comes from the Geo Service's OGC API: each zone is a polygon, a list of longitude/latitude corner points.

Draw both on the same canvas using one simple conversion from longitude/latitude to screen x/y, and the warnings will line up with what is on screen, because the processor checks the aircraft against the same zones you are drawing.

A **coastline** is only there to help the eye. Start without one. Later, add a rough outline of the land, either a simplified real coastline or one you make up yourselves and serve as a collection like the zones. The zones and scenarios are fictional, so nothing has to be geographically accurate, and nothing in the core depends on the coastline. Real map tiles underneath are an optional expansion (E9).

VD-FUN-040 (Map) The console shall draw a coastline (simplified or made-up is fine; see above), all zones (active and inactive visibly different), avoidance areas (toggleable), all tracks with their heading, and each track's projected path. Tracks shall be coloured by their highest open warning level.

VD-FUN-041 (Live Feed) The console shall receive tracks, alerts, proposals, decisions and acknowledgements live from Redis (read-only), on a **worker thread**, passing updates to the UI with signals. The main thread shall never block on Redis.

VD-FUN-042 (Warning Panel) A list of open alerts sorted by level then time to entry, showing track, zone, level, time to entry and acknowledgement state. Operators and supervisors shall be able to acknowledge from the list.

VD-FUN-043 (Vektor Proposals) Proposed detours shall be drawn as dashed lines on the map with extra distance and time. Operators and supervisors shall be able to approve or reject a proposal; observers shall see it without those actions.

VD-FUN-044 (Drill-Down) Selecting a track shall show its details and its last **10 minutes** of history from the Geo Service's history API.

VD-FUN-045 (Zone Control) Supervisors shall be able to enable or disable a zone from the console.

VD-FUN-046 (Login) The console shall log in through Keycloak and send the access token with every protected request. The browser-based Authorization Code flow with PKCE (e.g. Qt Network Authorization) is the target. The Resource Owner Password flow may be used as an interim step if the switch is recorded in the decision log.

VD-FUN-047 (Role-Aware UI) Actions a role cannot perform shall be hidden or disabled. Services shall still reject them (VD-SEC-002).

3.2.7 Geo Service — Storage

VD-FUN-050 (Track Storage) The Geo Service shall consume `varde:tracks` via a consumer group and store positions in a **TimescaleDB hypertable** with a PostGIS point column and a spatial index. Inserts shall be batched.

VD-FUN-051 (No Gaps on Restart) If the Geo Service restarts, it shall resume from its consumer group position so no stored history is lost for events still in the stream.

3.2.8 Ops Service — Alerts, Approvals and Audit

VD-FUN-060 (Alert Store) The Ops Service shall consume `varde:alerts` and keep each alert's current level, open/closed state and acknowledgement.

VD-FUN-061 (Acknowledgement) Acknowledging records who and when, and publishes an [AlertAcknowledgedEvent](#). **A rise in level clears the acknowledgement** (the operator must acknowledge again); a fall keeps it.

VD-FUN-062 (Proposal Workflow) The Ops Service shall track proposal states:

From	Event	To
—	VektorProposalEvent received	PROPOSED
PROPOSED	operator/supervisor approves	APPROVED
PROPOSED	operator/supervisor rejects	REJECTED
PROPOSED	60 s without a decision	EXPIRED
PROPOSED	its alert CLOSES	EXPIRED
PROPOSED	a newer proposal for the same track arrives	EXPIRED

Every transition shall publish a [VektorDecisionEvent](#).

VD-FUN-063 (Audit Trail) The Ops Service shall store every acknowledgement, approval, rejection and every [AuditEvent](#) from the Geo Service, and serve them to supervisors (VD-IF-020).

3.2.9 Feature Expansion

After the core pipeline is complete, consultants can select and implement:

E1 — Push Zone Changes The Geo Service publishes a [ZoneSetChangedEvent](#) on every zone change; the processor refetches immediately. VD-PRF-003 tightens to **2 s**.

E2 — WebSocket Live Feed The Ops Service serves the live feed to Qt over an authenticated WebSocket, so the console no longer connects to Redis.

E3 — Web Observer Map A read-only web map (Leaflet, Angular or React) built on the OGC API, showing zones and recent tracks.

E4 — Altitude Bands Zones gain floor and ceiling altitudes; warnings consider altitude and vertical speed. Removes the overflight false warnings (2.6).

E5 — Replay Play back a chosen past period from TimescaleDB on the Qt map, with play, pause and speed controls.

E6 — Statistics Warnings per zone per hour/day, pre-computed with TimescaleDB continuous aggregates and shown in the console.

E7 — Advanced OGC CQL2 filtering (e.g. [filter=type='DANGER'](#)) and/or passing the OGC conformance test suite for Part 1.

E8 — Zone Editing in QGIS OGC API – Features Part 4 (create, replace, delete) on [restricted-areas](#), protected by the Supervisor role, so zones can be drawn in QGIS.

E9 — Basemap Tiles Map tiles behind the Qt map (e.g. an OpenStreetMap-based tile service, respecting its usage policy).

E10 — Non-Convex Detours Detours around non-convex avoidance areas (e.g. a visibility-graph search), instead of the convex hull.

E11 — Supervisor Web Console (Angular) An Angular web app for supervisors, logging in through Keycloak (Authorization Code + PKCE) and reading the Ops Service API:

- **Audit trail** with filtering (actor, role, action, zone, track, time range), paging and sorting, and CSV export.
- **Decision history:** every Vektor proposal with its outcome (APPROVED / REJECTED / EXPIRED), who decided, how long it waited, and extra distance and time.
- **Alert statistics:** alerts per zone and level, time from PREDICTED to acknowledgement.
- Supervisor role only. Operators and observers are refused by the **Ops Service** (403), not just hidden in the UI.
- The Ops Service's audit endpoint shall support the filters and paging server-side. The Angular app shall not download the whole trail and filter it in the browser.
- The decision log shall compare the Angular login with the Qt login (VD-FUN-046). If E3 is also chosen, both may live in the same Angular app.

E12 — Linux VM Deployment Deploy the full backend Compose stack (Redis, PostgreSQL, Keycloak, processor, Geo Service, Ops Service) to a Linux VM in the cloud (e.g. Azure), administered over SSH, with the Qt console connecting to it from an operator's machine:

- Only the ports the console and QGIS need are open; secrets live in an `.env` file on the VM, never in the repository.
- Keycloak and the services are configured for the VM's address instead of `localhost`.
- Deployment and update steps are documented (VD-CI-004), manual or scripted. Automating the update from CI is a bonus.
- A good fit once the Geo Service and Ops Service are stable: it is shared work for whoever owns those services.

3.3 Performance Requirements

VD-PRF-001 (Cycle) — The processor cycle shall stay at 500 ms $\pm$ 10 % with scenario S4 (150 tracks) and at least 6 zones. Proximity and detour computation per cycle shall be **measured** and reported.

VD-PRF-002 (Warning Latency) — A level change shall appear in the Qt console within **1 s** of the cycle that computed it.

VD-PRF-003 (Zone Change Latency) — A zone enabled or disabled by a supervisor shall affect warnings within **30 s** (VD-FUN-012).

VD-PRF-004 (History Query) — The history API shall return 10 minutes of one track within **500 ms** after the system has stored at least 1 hour of S4 traffic.

VD-PRF-005 (UI Responsiveness) — The Qt console shall remain responsive during continuous updates.

3.4 Data Requirements

VD-DAT-001 (Core Entities) — Entities equivalent to: [Track](#), [Zone](#), [AvoidanceArea](#), [Alert](#), [VektorProposal](#), [AuditRecord](#).

VD-DAT-002 (Sensitivity) — No companion policy applies. Aircraft identifiers come from public broadcasts and are treated as operational data. **Audit records identify staff**; they shall be readable only by supervisors and shall not be exposed through the OGC API.

VD-DAT-003 (Coordinate Order) — GeoJSON and PostGIS use longitude, latitude. Every conversion point shall be covered by a test (VD-TST-012).

3.5 Security Requirements

VD-SEC-001 (Identity) — A Keycloak realm [varde](#) with realm roles [operator](#), [supervisor](#), [observer](#), and at least one test user per role. The realm configuration shall be exported and committed so the environment can be recreated.

VD-SEC-002 (Server-Side Enforcement) — The Geo Service and Ops Service shall validate tokens and enforce roles on every protected endpoint.

VD-SEC-003 (Open Reads) — OGC read endpoints may be unauthenticated on the internal network so QGIS can connect. This decision shall be recorded.

VD-SEC-004 (Input Validation) — Every API shall validate its parameters ([bbox](#), [datetime](#), [limit](#), IDs, payloads) and never crash on bad input.

VD-SEC-005 (Secrets) — No secrets in any repository. Configuration via environment variables or files excluded from version control.

3.6 Software System Attributes

VD-REL-001 (Reliability) — Every component shall survive another component restarting, and shall reconnect to Redis, PostgreSQL and Keycloak automatically.

VD-MNT-001 (Maintainability) — The Varde and Vektor engines shall be separate modules with no Redis, Qt or network code, so they can be unit-tested in isolation.

VD-POR-001 (Portability) — All backend components run in Docker. The Qt console builds on at least one team OS, documented.

4. DevOps / CI/CD Recommendations

How to read Sections 4–6. These are **recommendations**, not a checklist. A team that tries to do all of them in 3.5 weeks will not reach an MVP. Use your discretion:

- Put CI, tests and documentation where they protect what matters most and is easiest to get wrong: the worked examples, geometry, the event contract and role enforcement.
- If there is a kind of testing or tooling here you have never used (Testcontainers, characterisation tests, generating Protobuf code in CI), try it at least once, for the exposure, even if you don't take it further.

- Record what you chose to do, what you skipped, and why, in the decision log (VD-DOC-005), and be ready to defend it in the presentation.

Items marked (*required*) stay required, because another part of this SRS or the deliverables depends on them: the GPL credit, as-found note and characterisation tests are legacy requirements (VD-LEG-001 to 003), and schema compatibility and coordinate order are data requirements (VD-IF, VD-DAT-003).

- **VD-CI-001 (CI per Component)** — Each component (processor, Geo Service, Ops Service, Qt console) should have CI that builds it and runs its tests on every merge request.
- **VD-CI-002 (Contract in CI)** — CI should generate Protobuf code from the shared `.proto` source for every language that uses it, so a contract change that breaks a consumer fails that consumer's build.
- **VD-CI-003 (Compose)** — One `docker compose up` should start Redis, PostgreSQL (PostGIS + TimescaleDB), Keycloak (with the committed realm), the processor, the Geo Service and the Ops Service.
- **VD-CI-004 (Deployment)** — If E12 is chosen, the backend should be deployed to a Linux VM over SSH, with the deployment steps (manual or automated) documented.
- **VD-CI-005 (Repositories)** — The processor stays in its fork. The team should decide whether the other components share one repository or have one each, and record the decision.

5. Documentation Recommendations

- **VD-DOC-001 (API Documentation)** — OpenAPI documentation for the Geo Service and Ops Service; the OGC API's `/api` definition.
- **VD-DOC-002 (README)** (*required*) — Project name, team members, architecture overview, credit to the inherited processor's authors, and instructions to run everything from scratch.
- **VD-DOC-003 (Architecture Diagram)** — A current architecture diagram showing components, streams and APIs.
- **VD-DOC-004 (Event Contract)** — Every event type, its producer and consumers, its stream, and the schema evolution rules.
- **VD-DOC-005 (Decision Log)** (*required*) — Significant decisions and their rationale, including at least: what was removed from the inherited processor and why, Streams vs Pub/Sub for the live feed, open OGC reads, the 2D limitation, the login flow, and repository structure.
- **VD-DOC-006 (Legacy Report)** (*required*) — The as-found note (VD-LEG-002) plus a short "what we changed and why" summary.

6. Verification and Testing Recommendations

6.1 Unit Testing

- **VD-TST-001** — Warning levels should be tested with every row of the worked example in VD-FUN-022, including the near miss.
- **VD-TST-002** — Point-in-polygon and segment-polygon intersection should be tested, including a path that only touches a vertex and a path along an edge.
- **VD-TST-003** — Hysteresis should be tested with the worked example in VD-FUN-023.
- **VD-TST-004** — Vektor side selection should be tested with the worked example in VD-FUN-031, and the detour should be checked not to enter the avoidance area.
- **VD-TST-005** — Scenario loading should be tested for determinism (same file → same tracks).
- **VD-TST-006** — The Ops Service proposal workflow should be tested for every transition in VD-FUN-062, including [409](#) on a late decision.
- **VD-TST-007** — Role enforcement should be tested: an observer approving, acknowledging or changing a zone gets [403](#).
- **VD-TST-008** (*required*) — A consumer generated from the version 2 schema shall parse a version 3 track snapshot.
- **VD-TST-009** (*required*) — Characterisation tests for inherited modules shall exist before those modules change (VD-LEG-003).

6.2 Integration Testing

- **VD-TST-010** — Processor → Redis Streams → Geo Service: stored track count matches published tracks for a scenario run.
- **VD-TST-011** — Processor → Redis → Ops Service: alerts opened, changed and closed in order for scenario S1.
- **VD-TST-012** (*required*) — OGC API: [bbox](#) filtering, paging ([next](#) links), [datetime](#) on tracks, [limit](#) bounds, and correct coordinate order in output.
- **VD-TST-013** — Geo Service restart during a scenario: no gap in stored history (VD-FUN-051).

6.3 Manual Verification Scenarios (demo script)

- **VD-TST-020 (Core pipeline)** — Run S1: the track turns PREDICTED, IMMINENT, then INFRINGEMENT on the Qt map, with the warning panel updating each time.
- **VD-TST-021 (Vektor approved)** — Run S3 as an operator: the dashed detour appears, approve it, the aircraft flies around the zone, and the alert closes.
- **VD-TST-022 (Vektor rejected)** — Run S3 again and reject: the aircraft continues into the zone and escalates to INFRINGEMENT, with no new proposal.
- **VD-TST-023 (Three roles)** — Show the same situation logged in as observer (no actions), operator (acknowledge, approve) and supervisor (plus zone control and audit trail).

- **VD-TST-024 (Zone control)** — As supervisor, disable the zone in S1 mid-run: its alerts close within 30 s. Run S5: enabling a zone over an aircraft gives INFRINGEMENT at the next zone refresh, with no PREDICTED or IMMINENT first.
- **VD-TST-025 (Standards)** — Connect QGIS to the OGC API and show zones, avoidance areas, coastline and recent tracks.
- **VD-TST-026 (Resilience)** — Stop the Geo Service mid-scenario: the processor keeps warning with its cached zones; restart it: history has no gap.

7. Deliverables

Consultants shall deliver:

1. The processor fork and the other component repositories, with clean commit histories.
2. The documentation the team chose from Section 5, including everything marked *(required)*.
3. The backend runnable with [docker compose up](#) (or deployed on a Linux VM, if E12 is chosen), and the Qt console runnable from documented steps.
4. A 30-minute presentation on **Thursday 29 October 2026**.

7.1 Presentation

Consultants must produce a 30-minute presentation which introduces the team and discusses their product:

- **Development (10 min).** Consultants must discuss the choices made during development, the difficulties they faced, and how they overcame them — including what they inherited, what they changed, and why.
- **Feature Demonstration (10 min).** Consultants must walk through the core functions of the system, highlighting how the system behaves differently for each user type and how each user type achieves their goals within the system.
- **Questions (10 min).** Consultants must be prepared to defend decisions made throughout development to the satisfaction of those present.

7.2 Suggested Milestones

By end of	Milestone
Week 1	Fork, as-found note and characterisation tests. Version 3 .proto agreed. Zones seeded in PostGIS and served over OGC (visible in QGIS). Qt map draws zones (coastline can follow later). Keycloak realm with roles. Compose running.
Week 2	Core pipeline (VD-TST-020). Streams consumers storing tracks and alerts. Acknowledgement working.
Week 3	Vektor proposals, approval workflow and simulated detours. Qt login. Extensions under way (e.g. VM deployment, E12).

Appendix A — Example Seed Zone

```
{
  "type": "Feature",
  "id": "D-101",
  "geometry": {
    "type": "Polygon",
    "coordinates": [[[5.00, 59.95], [5.20, 59.95], [5.20, 60.05], [5.00, 60.05],
[5.00, 59.95]]]
  },
  "properties": {
    "name": "Hjelteskjær Firing Range",
    "type": "DANGER",
    "enabled": true,
    "activeFrom": null,
    "activeTo": null,
    "marginNm": 3.0
  }
}
```

Note the coordinate order: **[longitude, latitude]**.